# Computer Vision & Deep Learning I

## CMSC 178IP - Digital Image Processing

Noel Jeffrey Pinton

University of the Philippines - Cebu
Department of Computer Science

# Course Outline

# Introduction to Neural Networks

# What is Deep Learning?

**Definition**

Deep Learning is a subset of machine learning based on artificial neural networks with multiple layers that progressively extract higher-level features from raw input.

**Key Characteristics:**

- Hierarchical feature learning
- End-to-end learning from data
- Automatic feature extraction
- Scalability with data and compute

**Applications in Computer Vision:**

- Image classification
- Object detection and localization
- Semantic segmentation
- Image generation and enhancement

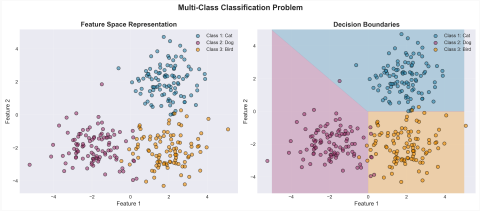
# Evolution of Computer Vision

**Traditional Approach:**

- Hand-crafted features (SIFT, HOG)
- Classical ML classifiers (SVM, RF)
- Limited representation capacity
- Manual feature engineering

**Deep Learning Approach:**

- Learned hierarchical features
- End-to-end optimization
- Superior performance
- Data-driven representations




Modern deep learning classification pipeline

# Perceptrons and Neural Networks

# The Perceptron

## Perceptron Model

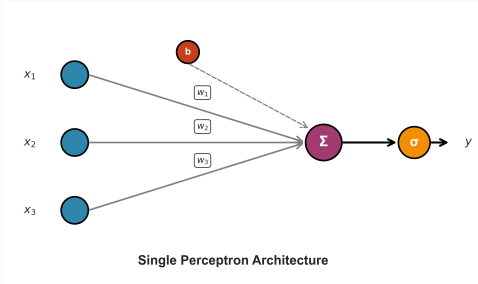
The perceptron is the fundamental building block of neural networks.

**Mathematical Formulation:**

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right) = f(\mathbf{w}^T \mathbf{x} + b)$$

Where:

- $\mathbf{x} = [x_1, x_2, \ldots, x_n]^T$ = input features
- $\mathbf{w} = [w_1, w_2, \ldots, w_n]^T$ = weights
- $b$ = bias term
- $f(\cdot)$ = activation function




Perceptron architecture with inputs, weights, and activation

# Perceptron Learning Rule

**Training Algorithm:**

1. Initialize weights $\mathbf{w}$ randomly
2. For each training sample $(\mathbf{x}, t)$:
   - Compute output: $y = f(\mathbf{w}^T\mathbf{x} + b)$
   - Calculate error: $e = t - y$
   - Update weights: $\mathbf{w} \leftarrow \mathbf{w} + \eta e\mathbf{x}$
   - Update bias: $b \leftarrow b + \eta e$
3. Repeat until convergence

**Limitation**

The perceptron can only learn linearly separable patterns. Complex problems require multi-layer networks.

# Multi-Layer Perceptron (MLP)

**Architecture Components:**

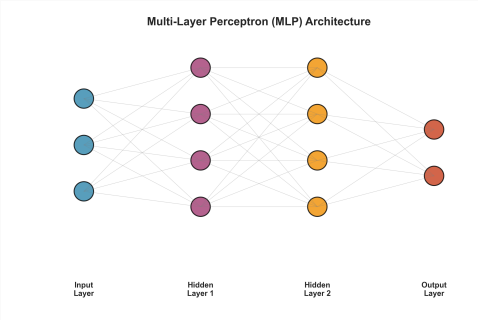
- **Input layer:** Raw features
- **Hidden layers:** Feature transformations
- **Output layer:** Final predictions

**Forward Propagation:**

$$\mathbf{h}_1 = f_1(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) \quad (1)$$

$$\mathbf{h}_2 = f_2(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2) \quad (2)$$

$$\mathbf{y} = f_3(\mathbf{W}_3\mathbf{h}_2 + \mathbf{b}_3) \quad (3)$$




Multi-layer perceptron with hidden layers

# Universal Approximation Theorem

**Key Theoretical Result**

A feedforward network with a single hidden layer containing a finite number of neurons can approximate any continuous function on compact subsets of $\mathbb{R}^n$, under mild assumptions on the activation function.

**Implications:**

- Neural networks are universal function approximators
- Sufficient network capacity can learn any pattern
- Depth enables efficient representation
- More layers = more expressive power

**In Practice:**

- Deep networks learn hierarchical representations
- Lower layers capture low-level features
- Higher layers capture abstract concepts

# Activation Functions

# Role of Activation Functions

**Purpose**

Activation functions introduce non-linearity into neural networks, enabling them to learn complex patterns beyond linear transformations.

**Key Properties:**

- **Non-linearity:** Essential for learning complex mappings
- **Differentiability:** Required for gradient-based optimization
- **Range:** Controls output magnitude
- **Monotonicity:** Affects optimization dynamics

**Without Activation Functions:**

$$\mathbf{y} = \mathbf{W}_2(\mathbf{W}_1\mathbf{x}) = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x} = \mathbf{W}\mathbf{x}$$

Multiple layers collapse to a single linear transformation!

# Common Activation Functions

**Sigmoid:**

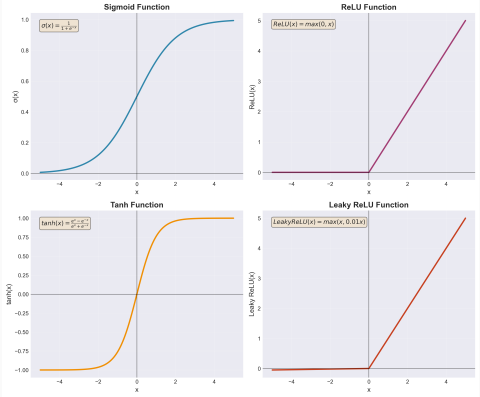
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Range: $(0, 1)$
- Smooth gradient
- Prone to saturation

**Hyperbolic Tangent (tanh):**

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

- Range: $(-1, 1)$
- Zero-centered
- Also saturates




Comparison of activation functions

# Modern Activation Functions

**ReLU (Rectified Linear Unit):**

$$\text{ReLU}(x) = \max(0, x)$$

- Simple and efficient
- No saturation for $x > 0$
- Most popular activation

**Leaky ReLU:**

$$\text{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{otherwise} \end{cases}$$

- Prevents dying neurons
- Small slope $\alpha \approx 0.01$

**Advantages of ReLU:**

1. Computational efficiency
2. Mitigates vanishing gradients
3. Better gradient flow

**Variants:**

- **PReLU:** Learnable $\alpha$
- **ELU:** Exponential for $x < 0$
- **Swish:** $x \cdot \sigma(x)$

# Loss Functions

# Loss Functions in Deep Learning

**Definition**

A loss function (or cost function) quantifies the difference between predicted outputs and ground truth labels, guiding the optimization process.

**General Form:**

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} L(y_i, \hat{y}_i) + \lambda R(\theta)$$

Where:

- $L(\cdot)$ = per-sample loss
- $y_i$ = ground truth label
- $\hat{y}_i$ = predicted output
- $R(\theta)$ = regularization term
- $\lambda$ = regularization strength

# Regression Loss Functions

**Mean Squared Error (MSE):**

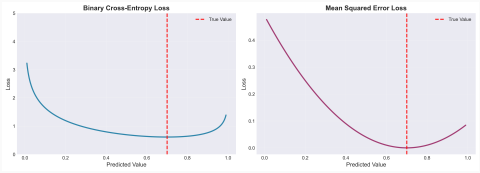
$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

- Differentiable everywhere
- Sensitive to outliers

**Mean Absolute Error (MAE):**

$$\mathcal{L}_{\text{MAE}} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

- Robust to outliers
- Linear penalty




Comparison of different loss functions

# Classification Loss Functions

**Binary Cross-Entropy:**

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$$

**Categorical Cross-Entropy:**

$$\mathcal{L}_{\text{CCE}} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c})$$

Where: $C$ = number of classes, $y_{i,c} = 1$ if sample $i$ is class $c$, $\hat{y}_{i,c}$ = predicted probability

**Softmax Output**

For multi-class: $\hat{y}_c = \frac{e^{z_c}}{\sum_{j=1}^{C} e^{z_j}}$

# Gradient Descent and Optimization

# Gradient Descent Fundamentals

**Basic Principle:**

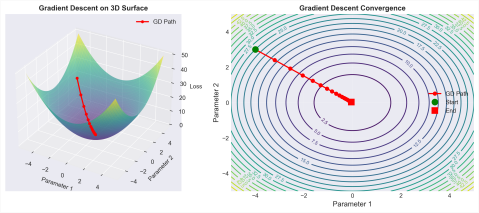
$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t)$$

Where:

- $\theta$ = model parameters
- $\eta$ = learning rate
- $\nabla_\theta \mathcal{L}$ = gradient of loss

**Variants:**

- **Batch GD:** Full dataset
- **Stochastic GD:** Single sample
- **Mini-batch GD:** Batch of samples




Gradient descent optimization trajectory

# Advanced Optimizers

**Momentum:**

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \nabla_\theta \mathcal{L}(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta \mathbf{v}_t$$

**Adam (Adaptive Moment):**

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t \quad (4)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2 \quad (5)$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \quad (6)$$

**Optimizer Comparison:**

- **SGD:** Simple, requires tuning
- **Momentum:** Faster convergence
- **RMSprop:** Adaptive learning rates
- **Adam:** Combines best of both

**Hyperparameters:**

- Learning rate: $\eta$
- Momentum: $\beta \approx 0.9$
- Adam: $\beta_1 = 0.9, \beta_2 = 0.999$

# Learning Rate Scheduling

**Why Schedule Learning Rates?**

- Start with large steps for fast initial progress
- Reduce step size for fine-tuning near optimum
- Prevent oscillations and divergence

**Common Schedules:**

**Step Decay:**

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/s \rfloor}$$

**Exponential Decay:**

$$\eta_t = \eta_0 \cdot e^{-\lambda t}$$

**Cosine Annealing:**

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\left(\frac{t\pi}{T}\right)\right)$$

# Backpropagation

# Backpropagation Algorithm

**Core Principle**

Backpropagation efficiently computes gradients of the loss function with respect to all network parameters using the chain rule of calculus.

**Chain Rule:**

$$\frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_i}$$

**Algorithm Steps:**

1. **Forward pass:** Compute activations and outputs
2. **Compute loss:** Evaluate error at output
3. **Backward pass:** Propagate gradients from output to input
4. **Update weights:** Apply gradient descent

# Backpropagation Mathematics

**For a single layer:**

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = f(z^{(l)})$$

**Gradient computation:**

$$\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial z^{(l)}} = \frac{\partial \mathcal{L}}{\partial a^{(l)}} \odot f'(z^{(l)})$$

**Weight gradients:**

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^T$$

**Recursive gradient:**

$$\delta^{(l-1)} = (W^{(l)})^T \delta^{(l)} \odot f'(z^{(l-1)})$$

# Computational Efficiency

**Why Backpropagation?**

**Naive approach:**

- Compute each gradient independently
- Complexity: $O(n \cdot m)$ where $n =$ parameters, $m =$ samples
- Redundant computations

**Backpropagation:**

- Reuse intermediate results
- Complexity: $O(m)$
- Single backward pass
- Memory-efficient

**Gradient Flow:**

- Forward: Input $\rightarrow$ Output
- Backward: Output $\rightarrow$ Input
- Automatic differentiation
- Modern frameworks handle this

**Challenges:**

- Vanishing gradients
- Exploding gradients
- Dead neurons

# Convolutional Neural Networks

# Introduction to CNNs

**Key Insight**

Convolutional Neural Networks leverage spatial structure in images through parameter sharing and local connectivity, dramatically reducing parameters while improving performance.

**Motivation for CNNs:**

- Images have spatial structure (nearby pixels are correlated)
- Fully-connected networks ignore this structure
- Too many parameters for high-resolution images
- Translation invariance is desirable

**CNN Principles:**

- **Local connectivity:** Each neuron sees only local region
- **Parameter sharing:** Same weights across spatial locations
- **Hierarchical features:** Low to high-level representations

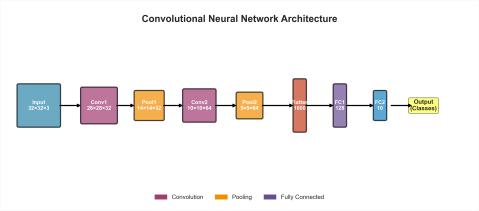
# CNN Architecture Overview

**Typical CNN Layers:**

1. **Convolution:** Feature extraction
2. **Activation:** Non-linearity (ReLU)
3. **Pooling:** Spatial downsampling
4. **Fully-connected:** Classification

**Information Flow:**

$$\text{Input} \rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Pool} \rightarrow \cdots \rightarrow \text{FC}$$




Complete CNN architecture

# Convolution Operation

**2D Convolution:**

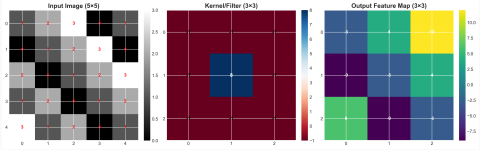
$$Y[i,j] = \sum_m \sum_n X[i+m, j+n] \cdot K[m,n]$$

Where:

- $X$ = input feature map
- $K$ = convolution kernel/filter
- $Y$ = output feature map

**Key Parameters:**

- **Kernel size:** $k \times k$ (e.g., 3×3, 5×5)
- **Stride:** Step size ($s$)
- **Padding:** Border handling




Convolution operation with kernel sliding over input

# Convolution Mathematics

**Output Dimensions:**

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - k}{s} \right\rfloor + 1$$

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2p - k}{s} \right\rfloor + 1$$

Where:

- $H, W$ = height, width
- $p$ = padding
- $k$ = kernel size
- $s$ = stride

**Padding Types:**

- **Valid:** No padding ($p = 0$), output size decreases
- **Same:** $p = \lfloor k/2 \rfloor$, output size preserved (for $s = 1$)

# Feature Maps and Filters

**Multiple Channels:**

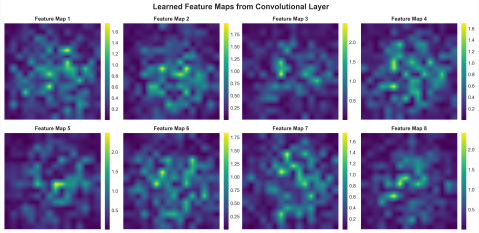
$$Y_k = \sum_{c=1}^{C_{\text{in}}} X_c \star K_{k,c} + b_k$$

**Parameters:**

- Input: $H \times W \times C_{\text{in}}$
- Kernel: $k \times k \times C_{\text{in}} \times C_{\text{out}}$
- Output: $H' \times W' \times C_{\text{out}}$

**Number of Parameters:**

$$\text{Params} = k^2 \cdot C_{\text{in}} \cdot C_{\text{out}} + C_{\text{out}}$$




Feature maps at different network depths

# Pooling Layers

# Pooling Operations

## Purpose of Pooling

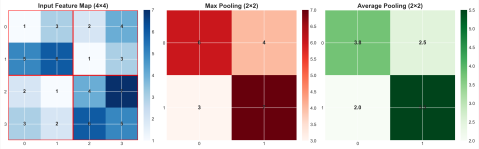
Pooling reduces spatial dimensions while retaining important features, providing translation invariance and computational efficiency.

**Max Pooling:**

$$Y[i,j] = \max_{m,n \in \mathcal{R}_{i,j}} X[m,n]$$

**Average Pooling:**

$$Y[i,j] = \frac{1}{|\mathcal{R}_{i,j}|} \sum_{m,n \in \mathcal{R}_{i,j}} X[m,n]$$




Max and average pooling operations

# Pooling Properties

**Advantages:**

- **Dimensionality reduction:** Reduces computation
- **Translation invariance:** Tolerates small shifts
- **Feature selection:** Max pooling keeps strongest activations
- **No parameters:** No learning required

**Common Configurations:**

- $2 \times 2$ window with stride 2 (most common)
- $3 \times 3$ window with stride 2
- Global average/max pooling (reduces to $1 \times 1$)

**Trends:**

- Modern architectures often use strided convolutions instead
- Global pooling popular for final feature aggregation

# CNN Architectures

# Classic CNN Architectures

**LeNet-5 (1998):**

- First successful CNN for digit recognition
- Conv $\rightarrow$ Pool $\rightarrow$ Conv $\rightarrow$ Pool $\rightarrow$ FC $\rightarrow$ FC

**AlexNet (2012):**

- ImageNet breakthrough
- 5 conv layers, 3 FC layers, ReLU, dropout

**VGGNet (2014):**

- Very deep (16-19 layers)
- Small $3\times3$ filters, simple architecture

**Key Insight:** Deeper networks with smaller filters outperform shallow networks.

# Modern CNN Architectures

**ResNet (2015):**

- Residual connections: $y = f(x) + x$
- Enables training very deep networks (50-152 layers)
- Solves vanishing gradient problem

**Inception/GoogLeNet (2014):**

- Multi-scale feature extraction
- Inception modules with parallel paths
- Efficient parameter usage

**EfficientNet (2019):**

- Compound scaling (depth, width, resolution)
- State-of-the-art efficiency
- Neural architecture search

# Training Deep Networks

# Training Pipeline

**Complete Training Loop:**

1. **Data loading:** Load mini-batch
2. **Forward pass:** Compute predictions
3. **Loss computation:** Calculate error
4. **Backward pass:** Compute gradients
5. **Parameter update:** Apply optimizer
6. **Evaluation:** Monitor metrics

**Hyperparameters:**

- Learning rate
- Batch size
- Number of epochs
- Optimizer choice


Training and validation learning curves

# Preprocessing Pipeline

**Essential Preprocessing:**

1. **Normalization:**

$$x' = \frac{x - \mu}{\sigma}$$

2. **Standardization:**

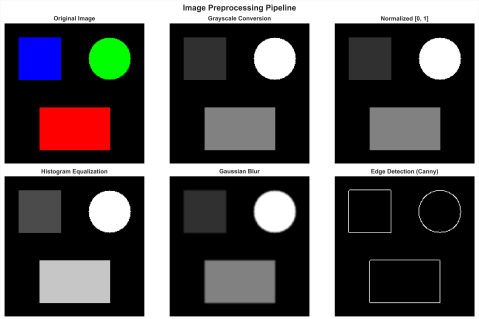
$$x' = \frac{x - \text{mean}}{\text{std}}$$

3. **Rescaling:**

$$x' = \frac{x - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}$$

**Common Schemes:**

- ImageNet mean/std
- Scale to [0,1] or [-1,1]




Preprocessing pipeline

# Batch Normalization

**Motivation:**

- Stabilize training and allow higher learning rates
- Reduce internal covariate shift

**Algorithm:**

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i \tag{7}$$

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2 \tag{8}$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \tag{9}$$

$$y_i = \gamma \hat{x}_i + \beta \tag{10}$$

Where $\gamma, \beta$ are learnable parameters.

# Overfitting and Regularization

# Understanding Overfitting

**Overfitting**

Model performs well on training data but poorly on test data due to memorizing noise.

**Causes:**

- Too many parameters
- Insufficient training data
- Lack of regularization

**Signs:**

- Large gap between train/val loss
- Decreasing val accuracy


Overfitting visualization

# Regularization Techniques

**L2 Regularization (Weight Decay):**

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda \sum_i w_i^2$$

**L1 Regularization:**

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda \sum_i |w_i|$$

**Dropout:**

- Randomly drop neurons during training (probability $p \approx 0.5$)
- Forces redundant representations
- Ensemble effect

**Early Stopping:** Stop training when validation loss increases

# Dropout in Detail

**Training Phase:**

$$h_i' = \begin{cases} 0 & \text{with probability } p \\ \frac{h_i}{1-p} & \text{with probability } 1-p \end{cases}$$

**Test Phase:** Use all neurons (no dropout), activations already scaled

**Benefits:**

- Prevents co-adaptation of neurons
- Implicit ensemble of networks
- Reduces overfitting significantly

**Typical Values:** 0.5 for hidden layers, 0.2-0.3 for input layers

# Data Augmentation

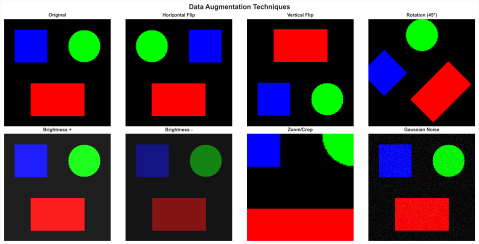
# Data Augmentation Strategies

**Purpose**

Data augmentation artificially increases training set size by applying label-preserving transformations, improving generalization.

**Geometric Transformations:**

- Random rotation
- Horizontal/vertical flips
- Random crops
- Scaling and zooming
- Shearing and translation




Various data augmentation techniques

# Advanced Augmentation

**Color-based Augmentation:**

- Brightness, contrast, saturation
- Hue shifts and noise injection

**Modern Techniques:**

- **Cutout:** Random rectangular occlusion
- **Mixup:** Linear interpolation between samples
- **CutMix:** Replace image regions between samples
- **AutoAugment:** Learned augmentation policies

**Best Practices:**

- Apply augmentation online during training
- Use task-appropriate transformations
- Combine multiple augmentations

# Transfer Learning

# Transfer Learning Basics

**Core Concept**

Transfer learning leverages knowledge learned from one task to improve performance on a related task, especially valuable with limited training data.

**Why Transfer Learning?**

- Training from scratch requires massive datasets
- Pre-trained models capture general features
- Faster convergence
- Better performance with less data

**Common Approach:**

1. Start with pre-trained model (e.g., ImageNet)
2. Remove final classification layer
3. Add new layers for target task
4. Fine-tune on target dataset

# Transfer Learning Strategies

**Feature Extraction:**

- Freeze pre-trained layers, train only new layers
- Fast and stable, good for small datasets

**Fine-tuning:**

- Unfreeze some/all layers
- Train with lower learning rate
- Better performance, requires more data

**Layer-wise Strategy:**

- Early layers: Generic features (edges, textures)
- Later layers: Task-specific features
- Fine-tune later layers first, then gradually unfreeze earlier layers

# Practical Transfer Learning

**Learning Rate Strategy:**

- Pre-trained layers: $\eta = 10^{-5}$ to $10^{-4}$
- New layers: $\eta = 10^{-3}$ to $10^{-2}$
- Use different learning rates for different layer groups

**Decision Tree:**

- **Small dataset + Similar task:** Feature extraction
- **Small dataset + Different task:** Fine-tune top layers
- **Large dataset + Similar task:** Fine-tune all layers
- **Large dataset + Different task:** Train from scratch or fine-tune extensively

**Popular Pre-trained Models:**

- ResNet, VGG, Inception (ImageNet)
- EfficientNet, MobileNet (efficiency-focused)

# Evaluation and Metrics

# Classification Metrics

**Confusion Matrix:**

- True Positives (TP)
- False Positives (FP)
- True Negatives (TN)
- False Negatives (FN)

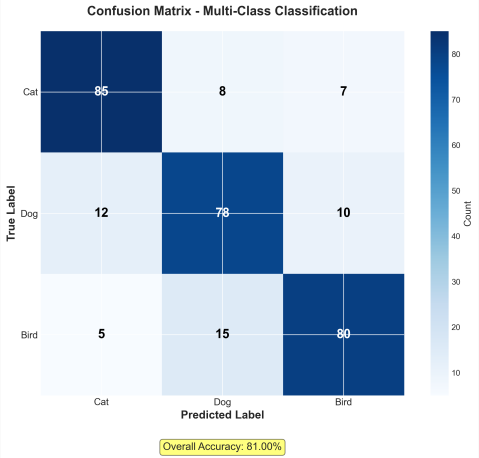
**Derived Metrics:**

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{11}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{12}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{13}$$

$$\text{F1} = 2 \cdot \frac{\text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}} \tag{14}$$




Confusion matrix for multi-class classification

# Model Evaluation Best Practices

**Data Splitting:**

- **Training set:** 60-80% for learning
- **Validation set:** 10-20% for tuning
- **Test set:** 10-20% for final evaluation

**Cross-validation:**

- K-fold CV for robust estimates
- Stratified sampling for imbalanced data
- Use when data is limited

**Monitoring:**

- Track train/val loss and accuracy
- Visualize learning curves
- Check for overfitting/underfitting
- Monitor gradient norms

# Best Practices

# Training Best Practices

**Initialization:**

- Use proper weight initialization (Xavier, He)
- Initialize biases to zero, never same values for all weights

**Optimization:**

- Start with Adam optimizer
- Use learning rate scheduling
- Gradient clipping for stability

**Regularization:**

- Apply dropout (0.5 for FC layers)
- Use weight decay ($\lambda = 10^{-4}$ to $10^{-5}$)
- Batch normalization for deep networks
- Data augmentation as primary regularizer

# Debugging Neural Networks - Part 1

**Common Issues and Solutions:**

**Loss not decreasing**

- Check learning rate (too high/low)
- Verify data preprocessing
- Check for bugs in loss computation

**Validation accuracy not improving**

- Model too simple (increase capacity)
- Poor hyperparameters
- Insufficient training time

# Debugging Neural Networks - Part 2

## NaN/Inf values

- Gradient explosion (reduce learning rate)
- Numerical instability (check activations)
- Division by zero (check loss function)

**Additional Debugging Tips:**

- Start with a small dataset to verify pipeline
- Check gradients are flowing properly
- Monitor activation distributions
- Validate data augmentation is working correctly

# Hyperparameter Tuning - Priority

**Priority Order:**

1. **Learning rate:** Most important
2. **Architecture:** Depth, width
3. **Batch size:** 32, 64, 128, 256
4. **Regularization:** Dropout, weight decay
5. **Optimizer:** Adam, SGD+Momentum

**Search Strategies:**

- **Grid search:** Systematic, exhaustive
- **Random search:** Often more efficient
- **Bayesian optimization:** Smart sampling
- **Manual tuning:** Domain expertise

# Learning Rate Finding

**Learning Rate Finding Technique:**

1. Start very small ($10^{-6}$)
2. Gradually increase learning rate
3. Stop when loss diverges
4. Use point before divergence

**Best Practices:**

- Use learning rate warmup for stability
- Consider cyclical learning rates
- Monitor both training and validation metrics
- Adjust based on gradient norms

# Summary

# Key Takeaways - Fundamentals

**Neural Network Basics:**

- Perceptrons are building blocks
- Activation functions enable non-linearity
- Backpropagation efficiently computes gradients
- Optimization guides learning process

**Loss and Optimization:**

- MSE for regression, Cross-entropy for classification
- Adam optimizer often best starting point
- Learning rate is critical hyperparameter
- Momentum accelerates convergence

# Key Takeaways - CNNs

**Convolutional Networks:**

- Leverage spatial structure of images
- Convolution extracts local features
- Pooling provides invariance and reduction
- Hierarchical feature learning

**Architecture Design:**

- Deeper networks learn better representations
- Residual connections enable very deep networks
- Balance between depth and width
- Pre-trained models accelerate development

# Key Takeaways - Training

**Practical Training:**

- Proper preprocessing is essential
- Data augmentation improves generalization
- Monitor train/val metrics closely
- Regularization prevents overfitting

**Best Practices:**

- Start simple, then increase complexity
- Transfer learning for limited data
- Systematic hyperparameter tuning
- Validate on held-out test set

# Next Steps in Deep Learning

**Advanced Architectures:**

- Attention mechanisms and Transformers
- Object detection (YOLO, R-CNN)
- Semantic segmentation (U-Net, DeepLab)

**Advanced Topics:**

- Generative models (GANs, VAEs, Diffusion)
- Self-supervised and few-shot learning
- Neural architecture search

**Applications:**

- Medical image analysis and autonomous driving
- Face recognition and biometrics
- Image generation and editing

# Questions & Discussion

**Ready to build your first CNN?**
**Check out the interactive notebook!**